

Experiment 5: Dynamic React Form Using Configuration Object

App.js

```
import React, { useState } from "react";

function App() {
  const fields = [
    { name: "name", label: "Name" },
    { name: "email", label: "Email" },
    { name: "phone", label: "Phone" }
  ];

  const [formData, setFormData] = useState({});
  const [submitted, setSubmitted] = useState(null);

  const handleChange = (e) => {
    setFormData({
      ...formData,
      [e.target.name]: e.target.value
    });
  };

  const handleSubmit = (e) => {
    e.preventDefault();
    setSubmitted(formData);
  };

  return (
    <div>
      <h2>Dynamic Form</h2>

      <form onSubmit={handleSubmit}>
        {fields.map((field) => (
          <div key={field.name}>
            <label>{field.label}</label>
            <input
              type="text"
              name={field.name}
              onChange={handleChange}
            />
          </div>
        ))}

        <button type="submit">Submit</button>
      </form>

      {submitted && (
        <div>
          <h3>Submitted Data</h3>
          <pre>{JSON.stringify(submitted, null, 2)}</pre>
        </div>
      )}
    </div>
  );
}

export default App;
```

Output

- The form fields are generated from the configuration object.
- After Submit, the entered data is displayed below the form.

Experiment 5: Dynamic React Form Using Configuration Object

Aim

1. To generate form fields dynamically using a JavaScript configuration object.
2. To manage form data using React state.
3. To display user-entered data after form submission.
4. To understand dynamic form rendering and event handling in React.

Procedure

1. Create a React application.
2. Create a JavaScript configuration object containing form field details.
3. Use `map()` to dynamically generate the form fields.
4. Use `useState()` to store form values.
5. Capture input using `onChange`.
6. Add a Submit button and display submitted data below the form.
7. Run using `npm start` and verify the form.

Sample Input

- Enter values in the dynamically generated Name, Email and Phone fields.
- Click Submit.

Sample Output

- The form fields are generated from the configuration object.
- After Submit, the entered data is displayed below the form.

Result

Thus, a Dynamic React Form was successfully developed using a JavaScript configuration object, React state and dynamic rendering.

Result

Thus, a Dynamic React Form was successfully developed using a JavaScript configuration object. The form fields were generated dynamically, user inputs were managed using React state, and the submitted data was displayed below the form without reloading the page.

Dynamic Form

Name

Email

Phone